

Introduction to Containers

IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

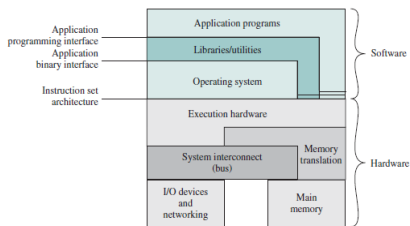
What is containerization?

How do computer programs work?

"OS driven Application Development" Operating Systems - Internals and Design Principles 7th ed - W. Stallings (Pearson, 2012)

- An OS has multiple layers of abstraction to allow smooth application development.
- A developer only needs to install "high level" libraries and utilities to develop apps.
- The OS takes care of everything under the hood to provide a seamless user experience.

What is happening under the hood?

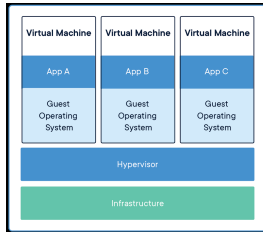


Simple python program

```
import numpy as np
a = np.array([1,2,3,4])
b = np.mean(a)
print(b)
```

- The python interpreter searches library dependencies in the installed libraries/utilities.
- Upon success, the relevant modules are imported in program runtime.
- The program logic statements are executed sequentially.
- The OS takes care of all the internals
 - How to interface with the processor for the math operation (addition for the mean).
 - Where to store the result?
 - How to interface with your monitor to "print" your command.

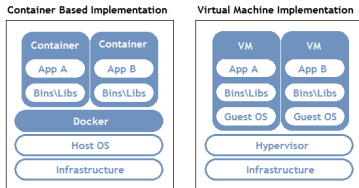
What are Virtual Machines (VM)?



<https://www.docker.com/>

- Since the early 2000s, there has been a push to squeeze more compute on physical machines
- Led to virtualization, wherein multiple host OS can be run on one physical system.
- On your Windows computer, download VMWare and now you can run Linux, AIX, HP-UX, BSD, FreeBSD etc.
- You can even run MacOS as well.

What are containers?

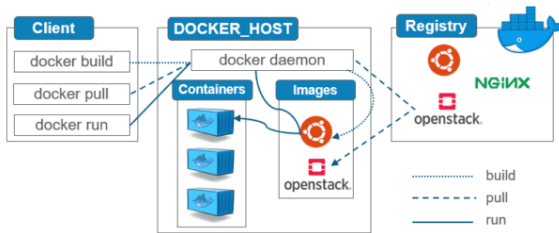


<https://www.aquasec.com/cloud-native-academy/docker-container/docker-architecture/>

- Containers go one step further by presenting a deeper level of virtualization.
- Abstracts away the painful process of installing and keeping track of system dependencies on host system.
- Allows rapid prototyping and roll out of software features faster.
- Docker and Podman are the popular choices.

How do containers work?

How do containers work?



<https://geekflare.com/docker-architecture/>

- Install docker daemon on development machines.
- Use Docker API to create container images and deploy them to a registry.
- These images can be used by anyone on any machine that supports docker.

Benefits of Containerization

Benefits of Containerization

Containerization enables the following capabilities.

- Sharing and running code without the pain of installing any dependencies.
- All software dependencies to be packaged into a container image by the research owners.
- Seamless execution of their container images by others.
- Layering of software frameworks without disruption.
- Support across any operating system! Linux, Windows even Unix!
- Container images are quick and easy to build and are the weapon of choice in DevOps Engineering!
- Popular containerization tools include Docker and Podman, which are mutually compatible 100%

Containerization Basics

Some Container Principles

- Container Creation
 - Hub and Base Images
- Container Repository
 - Create, Store and Share your code.
- Container Handling
 - Run and stop containers.
- Container Development
 - Debug, layer and reuse your code.



Docker Configuration and Install

- Download the docker client
 - <https://www.docker.com/products/docker-desktop>
- Install docker client
 - Make sure you have command line access.
- Create an account
 - The username and password is going to be required later.
- Windows users should rely on WSL (Windows Subsystem for Linux) or the Windows command line
 - <https://learn.microsoft.com/en-us/windows/wsl/install>
- Windows users, if you're unable to install Docker, please let me know as soon as possible.

How to create Docker containers?

Directory structure

- /path/to/code/
 - Dockerfile
 - requirements.txt
 - app.py

Dockerfile

```
FROM python:3.7-alpine
WORKDIR /code
RUN apk add --no-cache gcc musl-dev linux-headers
COPY /path/to/code/ .
RUN pip install -r requirements.txt
CMD ["python app.py"]
```

Annotations for the Dockerfile:

- `FROM python:3.7-alpine`: Docker image name to use as base
- `WORKDIR /code`: Change directory to location of code on the image
- `COPY /path/to/code/ .`: Copy code to correct location on the image
- `RUN pip install -r requirements.txt`: Install python dependencies for your program
- `RUN apk add --no-cache gcc musl-dev linux-headers`: Install dependencies if needed (not contained in base image)
- `CMD ["python app.py"]`: Command to run your code

- Create the image by running `docker build -t <app_name> .`
- Run the image by executing `docker run -d <app_name>`

Some base image examples



ubuntu

Official Image

Updated 20 days ago



fedora

Official Image

Updated 2 days ago

Official Docker builds of Fedora



mathworks/matlab

Verified Publisher

By MathWorks • Updated a month ago

Docker container for MATLAB



python

Official Image

Updated 9 days ago



mathworks/matlab-deps

Verified Publisher

By MathWorks • Updated 16 days ago

Dependencies for MATLAB and Simulink



intel/intel-optimized-pytorch

Verified Publisher

By Intel Corporation • Updated 2 months ago



intel/intel-optimized-tensorflow

Verified Publisher

By Intel Corporation • Updated 23 days ago



Explore more images at <https://hub.docker.com>

Creating a Docker Image

Execute these commands from `/path/to/code/`

- Run a simple build using

```
docker build .
```

Creating a Docker Image

Execute these commands from `/path/to/code/`

- Run a simple build using

```
docker build .
```

- Build and tag the image using

```
docker build -t home_example:1.0 .
```

Creating a Docker Image

Execute these commands from `/path/to/code/`

- Run a simple build using
`docker build .`
- Build and tag the image using
`docker build -t home_example:1.0 .`
- Inspect the image using
`docker image inspect home_example:1.0`

Some Container Principles

- Container Creation
 - Hub and Base Images
- Container Repository
 - Create, Store and Share your code.
- Container Handling
 - Run and stop containers.
- Container Development
 - Debug, layer and reuse your code.



Pushing to a Repository

Execute these commands from `/path/to/code/`

- Tag the image using

```
docker tag home_example:1.0 pramanan3/home_example:  
latest
```

Pushing to a Repository

Execute these commands from `/path/to/code/`

- Tag the image using

```
docker tag home_example:1.0 pramanan3/home_example:  
latest
```

- Push the image to a repository using

```
docker push pramanan3/home_example:latest
```

Pushing to a Repository

Execute these commands from `/path/to/code/`

- Tag the image using

```
docker tag home_example:1.0 pramanan3/home_example:latest
```

- Push the image to a repository using

```
docker push pramanan3/home_example:latest
```

- Remember!

- Registry is a service for many images and projects, such as `hub.docker.com`
- Repository is a collection of versions of the same image, such as the `home_example` repository, which contains `home_example:latest`, `home_example:v1.0`, and `home_example:debug`
- A registry stores a collection of repositories.
- Lets look at the registry and its repository we just used!

Some Container Principles

- Container Creation
 - Hub and Base Images
- Container Repository
 - Create, Store and Share your code.
- Container Handling
 - Run and stop containers.
- Container Development
 - Debug, layer and reuse your code.



Different Ways to Run a Container

- Running a container
 - A simple one-time run uses `docker run home_example:1.0`
 - Running as a daemon uses `docker run -d home_example:1.0`

Different Ways to Run a Container

- Running a container
 - A simple one-time run uses `docker run home_example:1.0`
 - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
 - `docker run -d --name create_daemon_ex home_example:1.0`

Different Ways to Run a Container

- Running a container
 - A simple one-time run uses `docker run home_example:1.0`
 - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
 - `docker run -d --name create_daemon_ex home_example:1.0`
- Here is a common gotcha that will fail. Why?
 - `docker run -d --name create_daemon_ex2 home_example:2.0`

Different Ways to Run a Container

- Running a container
 - A simple one-time run uses `docker run home_example:1.0`
 - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
 - `docker run -d --name create_daemon_ex home_example:1.0`
- Here is a common gotcha that will fail. Why?
 - `docker run -d --name create_daemon_ex2 home_example:2.0`
- Checking on containers
 - List running containers using `docker ps`
 - List all containers using `docker container ls -all`

Different Ways to Run a Container

- Running a container
 - A simple one-time run uses `docker run home_example:1.0`
 - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
 - `docker run -d --name create_daemon_ex home_example:1.0`
- This will fail. Why?
 - `docker run -d --name create_daemon_ex home_example:1.0`
- Checking on containers
 - List running containers using `docker ps`
 - List all containers using `docker container ls -a`
- Stop, remove, and rerun the container
 - `docker stop create_daemon_ex`
 - `docker rm create_daemon_ex`
 - `docker run -d --name create_daemon_ex home_example:2.0`
- SUCCESS!

Some Container Principles

- Container Creation
 - Hub and Base Images
- Container Repository
 - Create, Store and Share your code.
- Container Handling
 - Run and stop containers.
- Container Development
 - Debug, layer and reuse your code.



Let's use a different Dockerfile.

- `Dockerfile.debug`
- Note the difference! CMD `tail -f /dev/null`

Let's use a different Dockerfile.

- `Dockerfile.debug`
- Note the difference! CMD `tail -f /dev/null`

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -f Dockerfile.debug -t home_example:2.0 .
```

Debugging Containers

Let's use a different Dockerfile.

- Dockerfile.debug
- Note the difference! CMD tail -f /dev/null

Execute these commands from /path/to/code/

- Build and Tag using new Dockerfile

```
docker build -f Dockerfile.debug -t home_example:2.0 .
```

- Run Container as Daemon

```
docker run -d --name create_daemon_ex2 home_example  
:2.0
```

Debugging Containers

Let's use a different Dockerfile.

- Dockerfile.debug
- Note the difference! `CMD tail -f /dev/null`

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -f Dockerfile.debug -t home_example:2.0 .
```

- Run Container as Daemon

```
docker run -d --name create_daemon_ex2 home_example  
:2.0
```

- Explore the new container from within

```
docker exec -it create_daemon_ex2 /bin/sh
```

Today's Examples

You can find these examples at the following link.

https://github.com/paritoshpr/docker_tutorial

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -t home_layered_example:1.0 .
```

- Run Container as Daemon

```
docker run -d --name layered_ex1 home_layered_example  
:1.0
```

Layering Containers

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -t home_layered_example:1.0 .
```

- Run Container as Daemon

```
docker run -d --name layered_ex1 home_layered_example  
:1.0
```

- Let's try to debug this container

- Recall the Dockerfile.debug file from previous steps

```
docker build -f Dockerfile.debug -t  
home_layered_example:1.0  
docker run -d --name layered_ex2 home_layered_example  
:1.0
```

Layering Containers

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -t home_layered_example:1.0 .
```

- Run Container as Daemon

```
docker run -d --name layered_ex1 home_layered_example:1.0
```

- Let's try to debug this container

- Recall the `Dockerfile.debug` file from previous steps

```
docker build -f Dockerfile.debug -t home_layered_example:1.0
```

```
docker run -d --name layered_ex2 home_layered_example:1.0
```

- Explore the new container from within

```
docker exec -it layered_ex2 /bin/sh
```

- File and Data Handling
 - Consume data in files/folders from host.
 - Write or modify files/folders on host.
- Networking with Docker
 - Host Mode
 - Bridge Mode
- Bringing up services
 - Deploy and run many images.
 - Run multiple models at the same time.

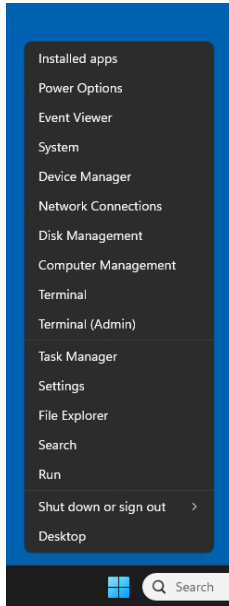
Today's Examples

You can find these examples at the following link.

https://github.com/paritoshpr/docker_tutorial

Windows Installation Instructions

Finding Your Device Specifications



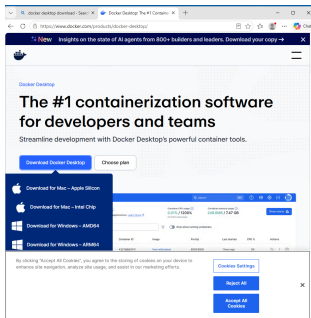
Checking System Type

The screenshot shows the 'Device specifications' window for a virtual machine named 'VDI-CEAT-19'. The window has a title bar with the VM name and version 'VMware20,1', and a 'Rename this PC' button. Below the title bar is a tab labeled 'Device specifications' with an information icon and a 'Copy' button. The main area contains a table of specifications:

Device name	VDI-CEAT-19
Full device name	VDI-CEAT-19.ad.okstate.edu
Processor	Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz (2.49 GHz) (2 processors)
Installed RAM	16.0 GB
Device ID	8EEE9C79-EC88-4B67-9E75-2C2FA53FEE13
Product ID	00329-00000-00003-AA758
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

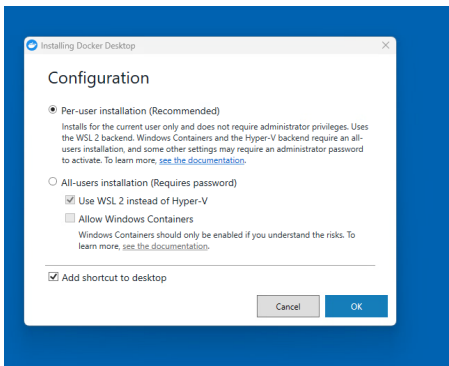
- Look at the **System type** field.
- "64-bit operating system, x64-based processor" corresponds to an **AMD64** machine, which covers both Intel and AMD chips.
- An ARM-based machine shows **ARM64** here instead.
- This determines which Docker Desktop installer you need.

Downloading Docker Desktop



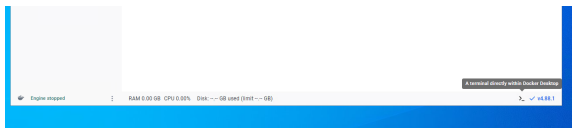
- Go to <https://www.docker.com/products/docker-desktop/>
- Choose the **Download for Windows AMD64** option for a standard x64 machine, or **Download for Windows ARM64** for an ARM-based machine.

Configuring the Installer



- **Per-user installation** is recommended and does not require administrator privileges.
- Keep **Use WSL 2 instead of Hyper-V** checked, matching the WSL guidance from earlier in this lecture.

After Installation



- Docker Desktop should show a running engine once installation finishes.
- An "**Engine stopped**" status usually points to a virtualization problem, most often that Intel VT-x/AMD-V is not enabled in the BIOS, or that the machine is a virtual desktop (VDI) that does not expose nested virtualization to the guest.
- On such machines, Play with Docker or GitHub Codespaces are practical fallbacks for coursework.